

第三节：流水线与冒险

五级流水线、时空图、暂停与转发

从单周期到流水线

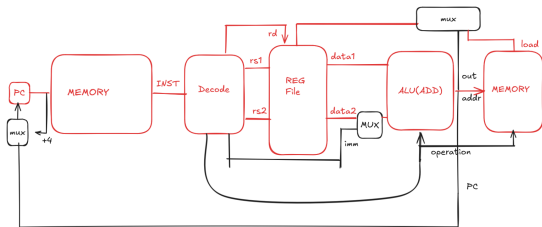
单周期设计把所有指令都放进一个被 lw 拉长的时钟周期。

- lw 经过取指、读寄存器、算地址、访存、写回。
- add 不访问数据内存，却也必须等待同样长的周期。
- 流水线把长路径切成几段，让多条指令在不同阶段重叠运行。

最长路径：lw

lw x5, 8(x6) 走过了单周期里的最长路径：

PC -> 指令内存 -> 解码 -> Reg[6] -> ALU -> 数据内存 -> Reg[5]



切分时按数据流经过的工作切，而不是按指令类型切。

五级流水线阶段

阶段	名称	对 lw x5,8(x6) 做的事	本段结束时留下的数据
IF ID	Instruction Fetch Instruction Decode	用 PC 读指令，同时算 PC+4 解码，读 Reg[6]，生成立即数 8	PC、instruction、PC+4 read_data1、imm、rd、控制信号
EX	Execute	ALU 计算地址 Reg[6]+8	alu_result、rd、访存/写回控制信号
MEM WB	Memory Access Write Back	数据内存读 DataMem[0x108] 把 mem_data 写入 Reg[5]	mem_data、rd、写回控制信号 无

流水线寄存器

段与段之间必须保存下一段还要用的数据。否则下一周期 PC 和 instruction 变化后，前一条指令的中间结果就丢了。

级间寄存器	要保存的内容	下一段为什么要用
IF/ID	PC、PC+4、instruction	ID 要解码这条指令；后续分支和顺序执行还要用这条指令自己的 PC 信息。

ID 阶段保存什么

instruction 被拆成寄存器号、立即数和控制信息。

级间寄存器	要保存的内容	下一段为什么要用
ID/EX	PC、PC+4、imm	EX 要计算分支目标或带立即数的地址。
ID/EX	read_data1、read_data2	EX 要把寄存器读出的操作数送入 ALU。
ID/EX	rs1、rs2、rd	后续冒险检测、转发选择和写回都要知道寄存器编号。
ID/EX	EX/MEM/WB 控制信号	当前指令自己的控制动作必须随指令向后流动。

EX/MEM/WB 的交接

级间寄存器	要保存的内容	下一段为什么要用
EX/MEM	branch_target、Zero	MEM 前后需要知道分支是否成立以及目标地址。
EX/MEM	alu_result、store_data、rd	MEM 要用地址和写内存数据；WB 还要知道可能写回的目标寄存器。
EX/MEM MEM/WB	MEM/WB 控制信号 mem_data、alu_result、rd	当前指令自己的访存/写回动作继续向后流动。 WB 要在内存结果和 ALU 结果之间选择，并写到正确寄存器。
MEM/WB	MemToReg、RegWrite	WB 要知道写回数据来源，以及是否真的写寄存器。

标准五阶段汇总

阶段	名称	输入寄存器	本阶段组合逻辑	输出寄存器
IF	Instruction Fetch	PC	指令内存读取、PC+4	IF/ID
ID	Instruction Decode	IF/ID	解码、读寄存器、立即数生成、控制信号生成	ID/EX
EX	Execute	ID/EX	ALU 运算、分支比较、分支目标计算	EX/MEM
MEM	Memory Access	EX/MEM	数据内存读写	MEM/WB
WB	Write Back	MEM/WB	选择 ALU 或内存结果，写寄存器堆	无

控制信号也要跟着流动

第 4 周期可能同时有四条不同指令分别在 ID、EX、MEM、WB 阶段。

关键点

WB 阶段要不要写寄存器，必须看正在 WB 的那条指令自己的 RegWrite，不能看 ID 阶段刚解码出来的新 RegWrite。

因此控制信号和数据一样，要被 ID/EX、EX/MEM、MEM/WB 保存并向后传递。

理想流水线时空图

指令/周期	1	2	3	4	5	6	7	8	9
I1	IF	ID	EX	MEM	WB				
I2		IF	ID	EX	MEM	WB			
I3			IF	ID	EX	MEM	WB		
I4				IF	ID	EX	MEM	WB	
I5					IF	ID	EX	MEM	WB

第 5 周期结束时 I1 写回；此后每个周期都有一条指令完成。

三类冒险

理想时空图会被三类情况打破：

类型	含义
结构冒险	两条指令在同一周期争用同一个硬件资源。
控制冒险	分支结果尚未确定，后续指令已经按顺序路径取入。
数据冒险	后一条指令需要前一条指令尚未写回的结果。

结构冒险：单端口存储器

例子

```
I1: lw x5, 0(x6)
I2: add x7, x8, x9
I3: sub x10, x11, x12
I4: addi x13, x14, 1
```

若指令内存和数据内存共用单端口存储器，I1 第 4 周期在 MEM 要读数据，I4 同一周期在 IF 要取指令。

同一个存储端口每周期只能响应一个请求。

结构冒险时空图

指令/周期	1	2	3	4	5	6
I1: lw	IF	ID	EX	MEM(读数据)	WB	
I2: add		IF	ID	EX	MEM	WB
I3: sub			IF	ID	EX	MEM
I4: addi				IF(取指)	ID	EX

解决办法：把指令存储和数据存储分成两块独立硬件，让 IF 和 MEM 各用各的端口。

控制冒险：分支结果晚到

例子

```
l1: beq x5, x6, 16  
l2: add x7, x8, x9  
l3: sub x10, x11, x12
```

设 $x5 = x6$ ，分支应跳到 $PC+16$ 。但比较结果要到 EX 阶段才出来，此时 l2、l3 已经按顺序地址被取入。

控制冒险时空图

周期	I1	I2	I3	发生的事
1	IF			取 beq
2	ID	IF		顺序取入 I2
3	EX	ID	IF	比较得知分支成立, I2、I3 走错路
4	MEM	flush	flush	PC 改为分支目标

被 flush 的指令不允许写任何状态。

数据冒险：读到旧值

例子

```
l1: add x5, x6, x7 # x5 = 42  
l2: sub x8, x5, x9 # 依赖 x5
```

设 $x6=20$ 、 $x7=22$ 、 $x9=2$ 。正确结果应是 $x5=42$ 、 $x8=40$ 。

什么都不做会错

指令/周期	1	2	3	4	5	6
I1: add x5,x6,x7	IF	ID	EX	MEM	WB	
I2: sub x8,x5,x9		IF	ID	EX	MEM	WB

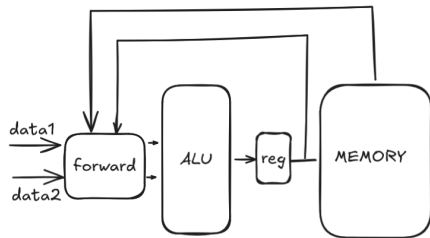
第 3 周期 I2 在 ID 读 x5，但 I1 的 x5 要到第 5 周期 WB 才写入寄存器堆。若旧 x5=0，I2 会算出 x8=-2。

暂停：等写回后再读

指令/周期	1	2	3	4	5	6	7	8
l1: add x5,x6,x7	IF	ID	EX	MEM	WB			
l2: sub x8,x5,x9		IF	ID(停)	ID(停)	ID	EX	MEM	WB
EX 中 bubble			bubble	bubble				

控制单元禁用 PC 和 IF/ID 写使能，ID/EX 控制信号清零。代价是流水线空转两个周期。

转发：从后级直接接回 EX



结果已经在流水线寄存器里，不一定要等它写回寄存器堆。

转发时空图

指令/周期	1	2	3	4	5	6
l1: add x5,x6,x7	IF	ID	EX(42)	MEM	WB	
l2: sub x8,x5,x9		IF	ID	EX(转发 42)	MEM	WB

第 4 周期，硬件发现 EX/MEM.rd == ID/EX.rs1 == 5，ForwardA 选择 EX/MEM 里的 42。

load-use: 转发也要等一拍

例子

I1: lw x5, 0(x6)

I2: add x7, x5, x8

加载数据要到 I1 的 MEM 阶段末尾才从内存读出来；I2 同一周期已经要在 EX 使用 x5。这时 EX/MEM 里还没有内存数据，无从转发。

load-use 时空图

指令/周期	1	2	3	4	5	6	7
I1: lw	IF	ID	EX	MEM(读数据)	WB		
I2: add		IF	ID	ID(停)	EX(转发)	MEM	WB
转发来源					从 MEM/WB 转发		

硬件检测到 load-use 后，让 I2 在 ID 多待一拍，EX 阶段空出一个 bubble。

顺序流水线的限制

例子

I1: lw x5, 0(x6)

I2: add x7, x5, x8 # 依赖 I1, 必须等待

I3: add x9, x10, x11 # 与 I1、I2 无关

顺序流水线按取指顺序处理指令：I2 因为依赖 x5 必须暂停，排在后面的 I3 即使源寄存器已就绪，也必须跟着停在 IF/ID。

顺序流水线限制的时空图

指令/周期	1	2	3	4	5	6	7	8
I1: lw x5,0(x6)	IF	ID	EX	MEM	WB			
I2: add x7,x5,x8		IF	ID	ID(停)	EX	MEM	WB	
I3: add x9,x10,x11			IF	IF(被挡住)	ID	EX	MEM	WB

I3 的 Reg[10] 和 Reg[11] 可能从一开始就有正确值，但它没有自己的等待位置，只能占着取指/解码入口排队。

顺序流水线解决了「一条指令太慢」的问题，但没有解决「一条指令等待时是否应该挡住无关指令」的问题。